VISUAL NOTES • COMPLETE GUIDE

Visual Notes — AI Code Review at Anthropic

Claude Writes 90% of Its Code. Here's How They Handle Review

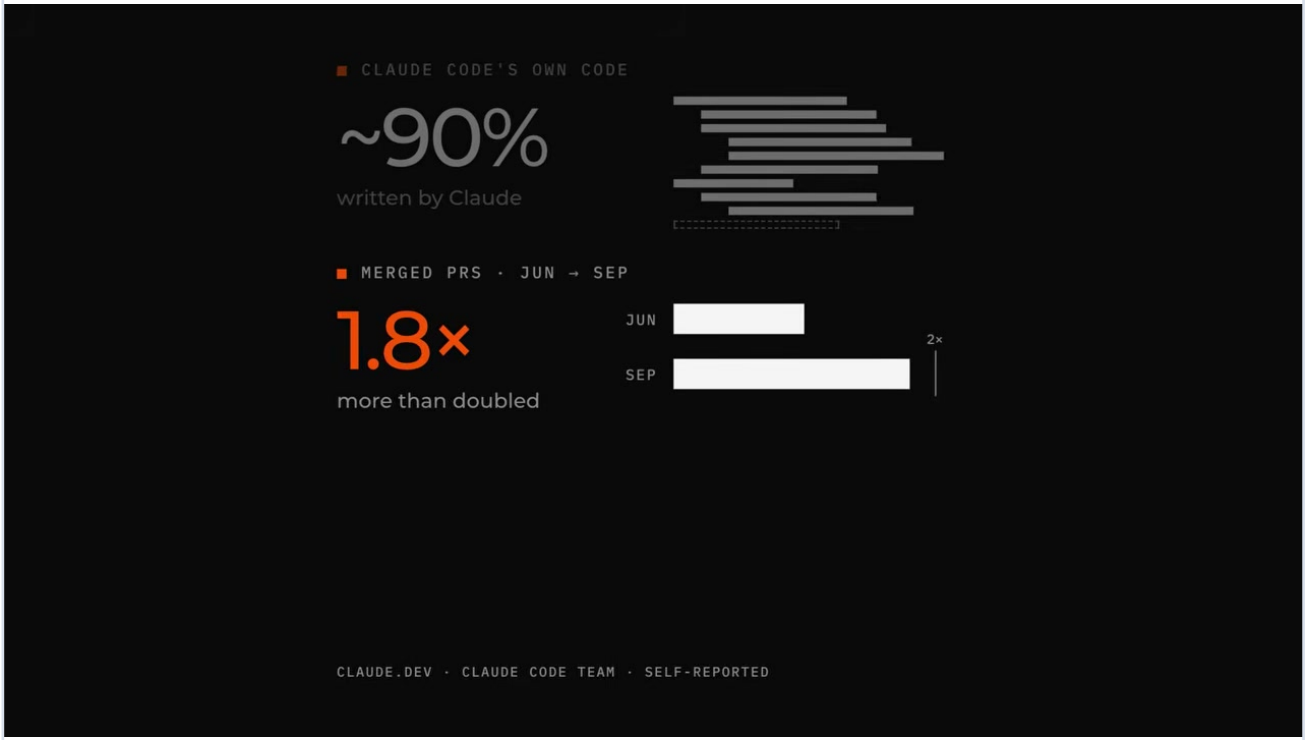
17:06

- 36 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=bls8ZJ_INMQ

More code merged, fewer reverts

Headline results — self-reported by Anthropic



NOTES

- Claude reportedly writes about 90% of Claude Code's own code.
- Merged pull requests more than doubled between June and September while reverts reportedly fell by more than 40%.
- The slide also shows a –25% share figure, so the precise revert metric is ambiguous.
- These are Anthropic's internal figures, not independently audited benchmarks.

CORE IDEA

Higher output matters only when failure and rollback rates do not rise.

Review moved; it was not removed



NOTES

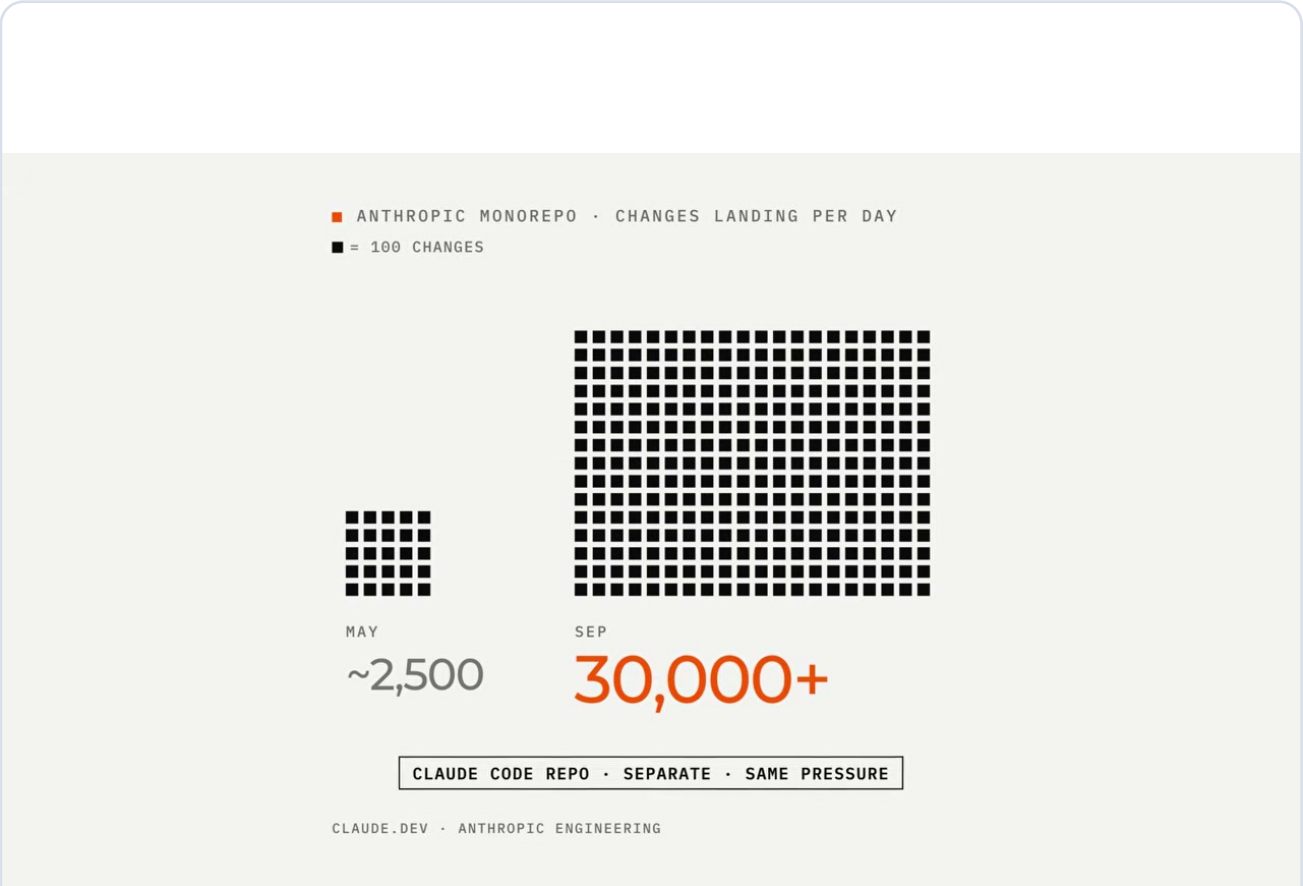
- Machines perform most of the reading that previously consumed reviewer attention.
- Humans remain responsible for judgment, accountability, and context that automated checks cannot supply.
- Fast recovery limits blast radius: the damage a bad change can cause before it is reversed.
- This is a design principle, not evidence that every team can remove the same review work.

CORE IDEA

Automate reading, reserve people for judgment, and make mistakes cheap to reverse.

Change volume rose from ~2,500 to 30,000+ per day

Anthropic monorepo scale — self-reported



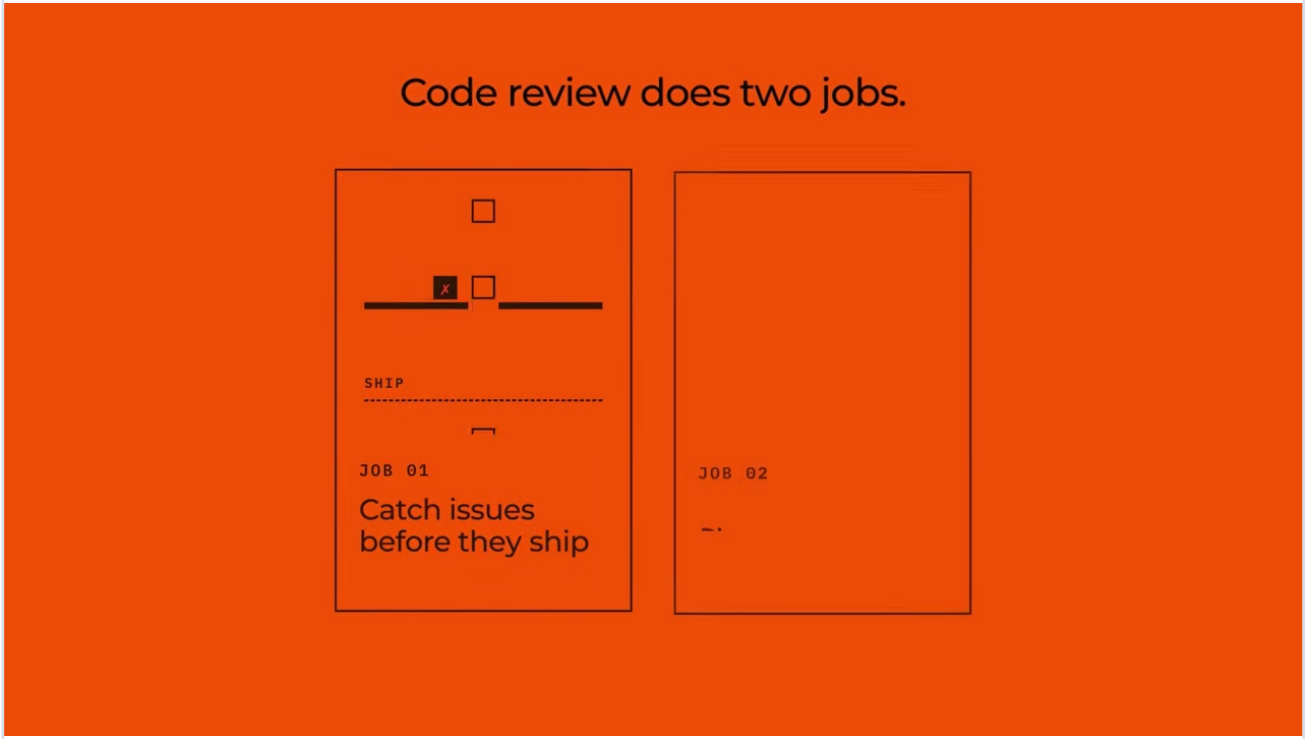
NOTES

- Daily changes landing in Anthropic's monorepo reportedly grew roughly twelvefold from May to September.
- A monorepo stores many projects in one repository, increasing the interaction surface between changes.
- At this volume, line-by-line human review becomes a throughput bottleneck.
- Anthropic's scale and compute budget are unusual, so the same architecture may be excessive for smaller teams.

CORE IDEA

Extreme change volume requires review systems designed for queues, not individual diffs.

Code review performs two different jobs



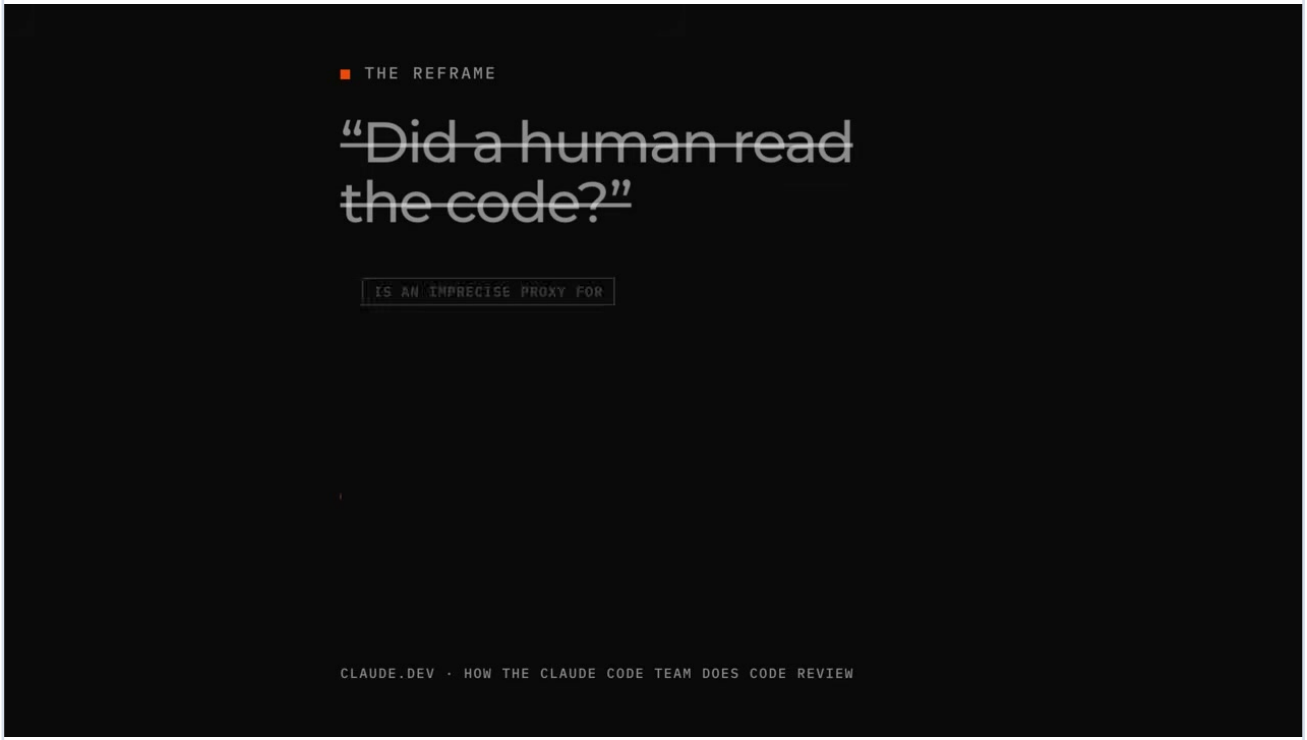
NOTES

- Review catches defects before deployment and spreads knowledge across the engineering team.
- Automated checks can cover much of defect detection, especially repeatable rules and known failure patterns.
- Shared context is harder to automate because reading teammates' code teaches architecture and local conventions.
- Replacing the first job without preserving the second can create a team that ships quickly but understands less.

CORE IDEA

Automating defect detection does not automatically preserve team knowledge.

Reframe review around risk



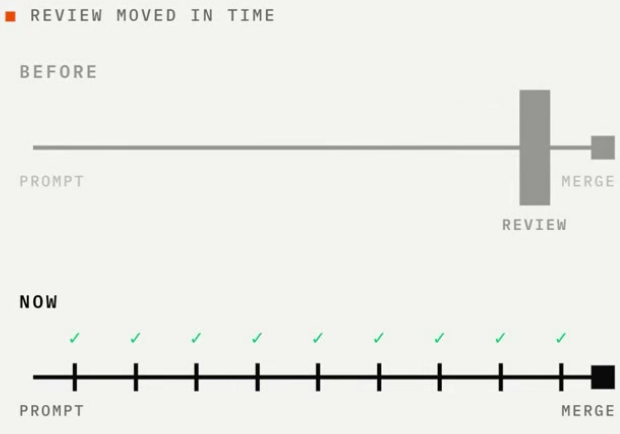
NOTES

- Whether a human read the code is only a proxy for whether important risks were examined.
- Start with what could fail, then assign the cheapest reliable automated or human check to each risk.
- The aim is to remove waiting and repetitive reading, not accountability.
- Risk classification must be revisited whenever incidents expose a blind spot.

CORE IDEA

Ask what can fail, then choose the check that reliably catches it.

Review becomes continuous feedback



NOTES

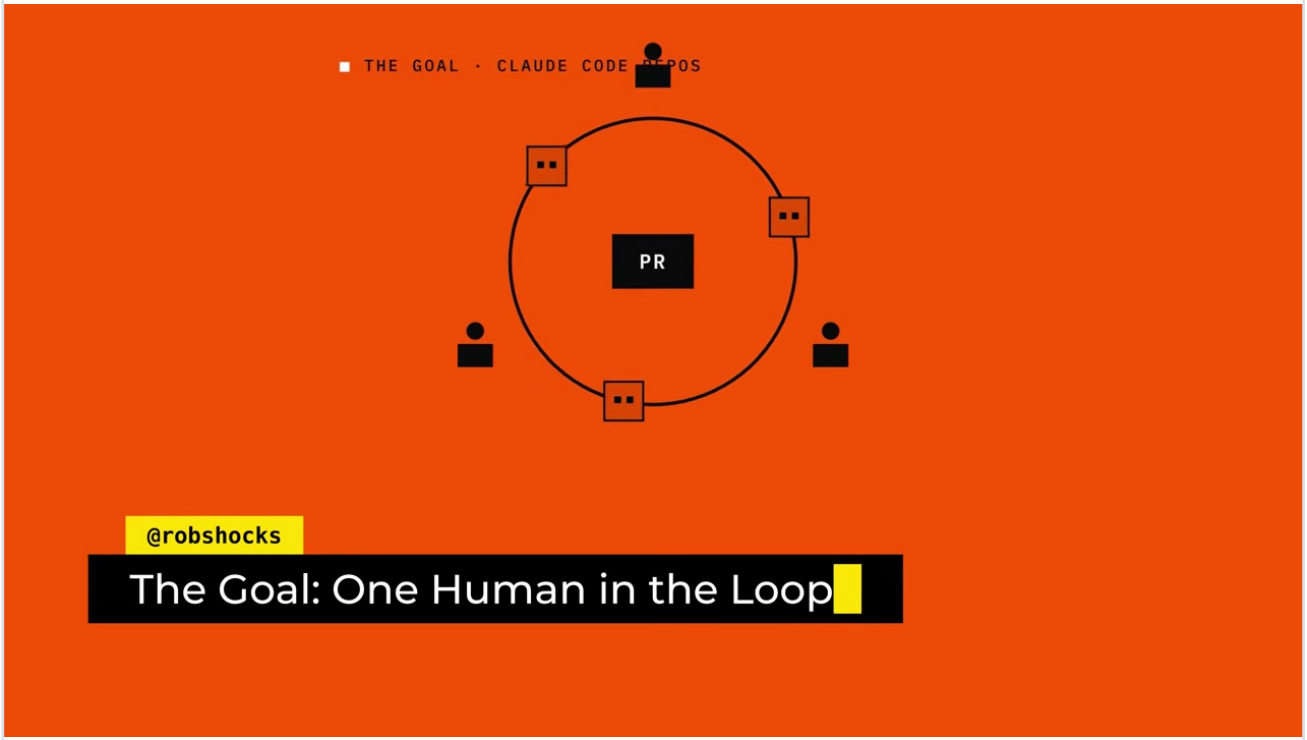
- The old process concentrates review near merge, creating a queue after most work is already finished.
- The new process spreads checks from the initial prompt through implementation and integration.
- The analogy is spell-check while writing: cheap feedback arrives before mistakes compound.
- Continuous checks still need an explicit final authority for high-risk changes.

CORE IDEA

Move checks earlier and distribute them across the development loop.

Source: https://www.youtube.com/watch?v=bls8ZJ_INMQ

Target: at most one accountable human per pull request



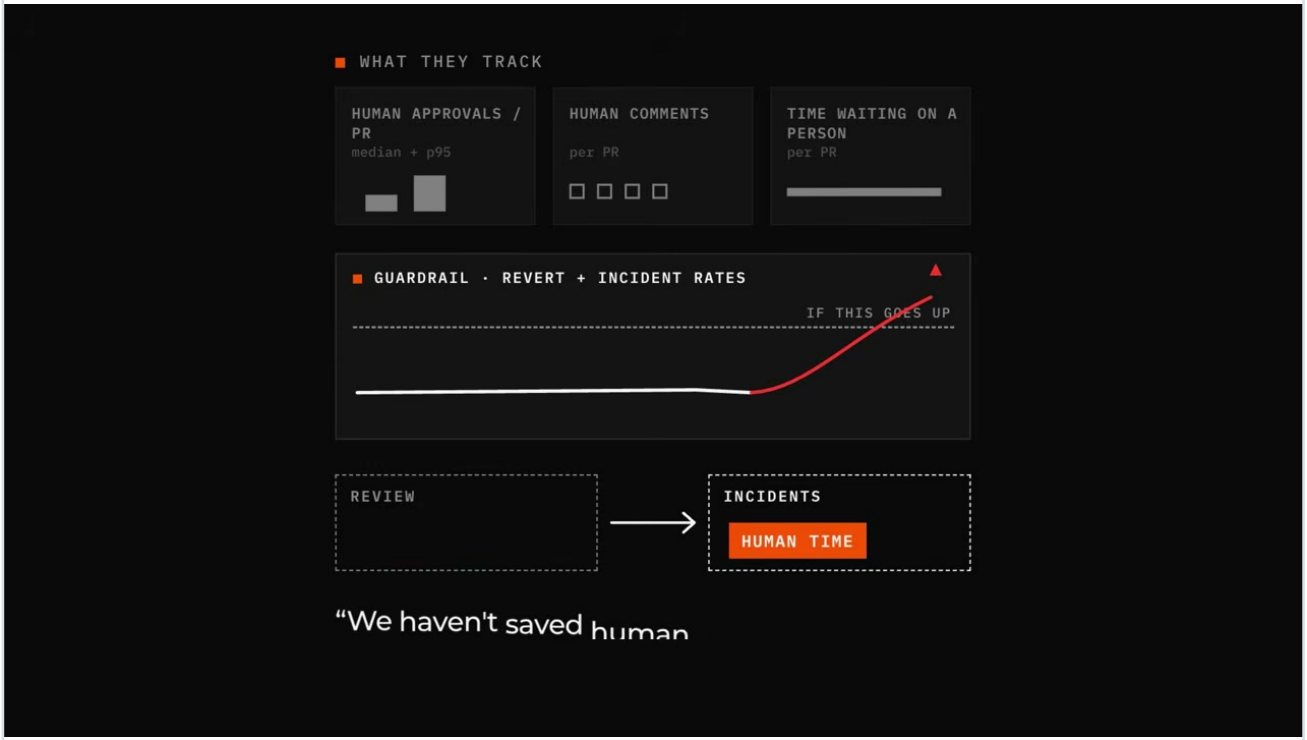
NOTES

- Most automated review activity surrounds the pull request before a person needs to intervene.
- The remaining human can be the author, replacing the traditional mandatory second pair of eyes.
- This is a stated target for Claude Code repositories, not a demonstrated outcome for every pull request.
- Risky changes still require stronger human approval later in the process.

CORE IDEA

Reduce human touches without removing human ownership.

Measure human effort and safety together



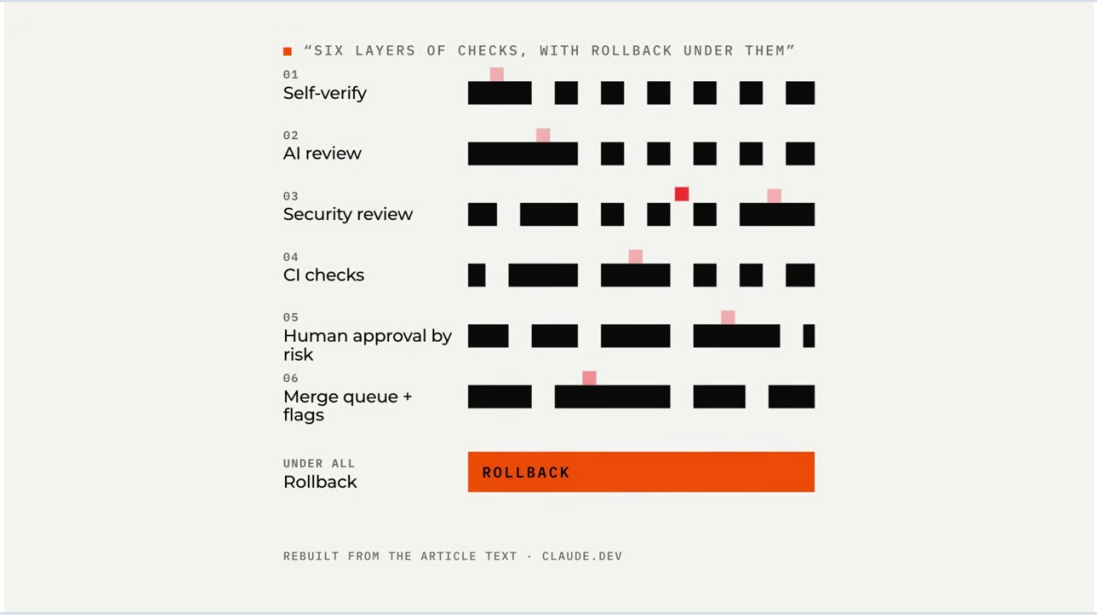
NOTES

- Operational metrics include human approvals, human comments, and time waiting on a person.
- Revert and incident rates act as guardrails against hollow productivity gains.
- If failures rise, automation may only have moved human work from review into incident response.
- The metric set should capture total human cost, not only pull-request latency.

CORE IDEA

Speed gains are real only when downstream failure work stays controlled.

Six layers of checks, with rollback underneath

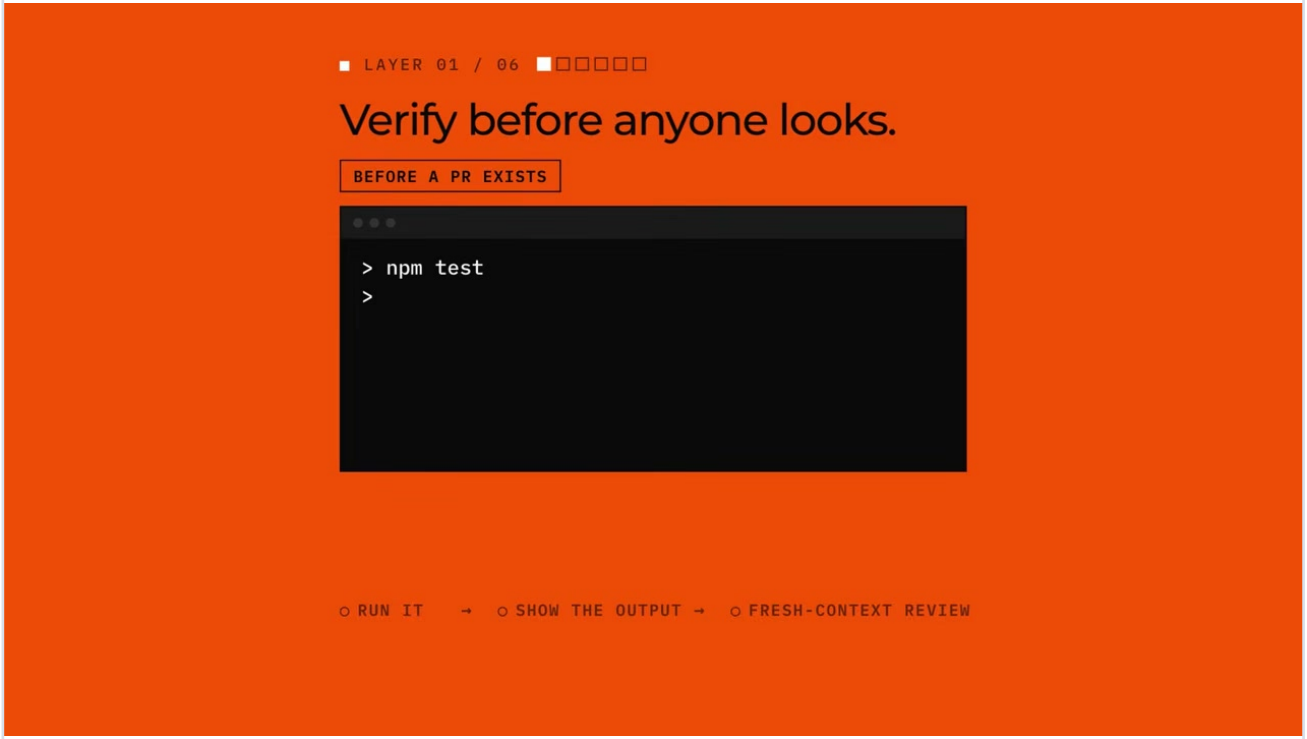


NOTES

CORE IDEA

Layer independent controls and keep recovery available when prevention fails.

Layer 1: verify before anyone looks



NOTES

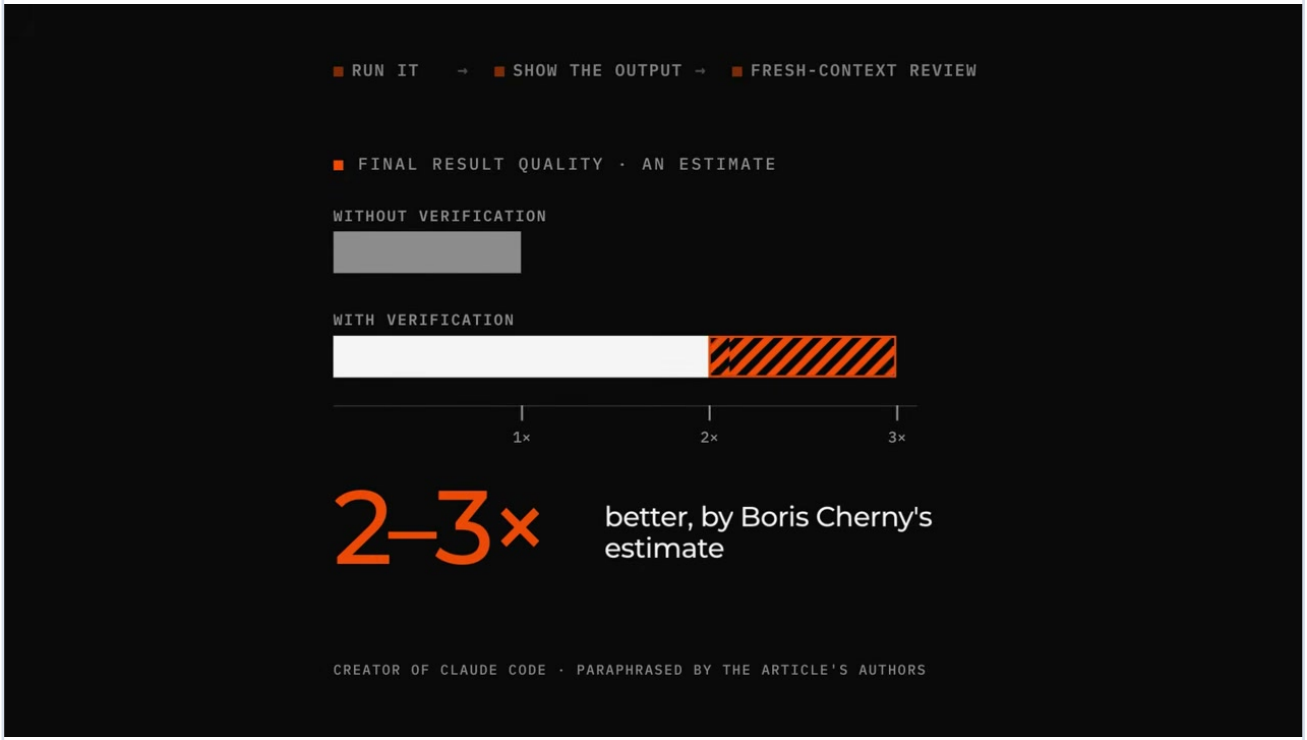
- The agent runs tests and builds, inspects screenshots, and executes end-to-end checks before opening a pull request.
- It must show outputs rather than merely claiming that verification succeeded.
- A fresh-context sub-agent reviews the diff without inheriting the implementer's assumptions.
- Verification quality depends on whether the repository exposes meaningful checks the agent can run.

CORE IDEA

Require executable evidence before spending reviewer attention.

Verification is estimated to improve results 2–3×

Individual estimate, not a measured benchmark



NOTES

- The workflow is: run the change, show the output, then review it with fresh context.
- Boris Cherny's estimate is that this produces results two to three times better.
- The claim is explicitly presented as an estimate and lacks a published evaluation method.
- Treat the multiplier as directional support for verification, not a portable performance guarantee.

CORE IDEA

Verification likely helps substantially, but the multiplier is only an estimate.

Cap nitpicks before they bury real findings



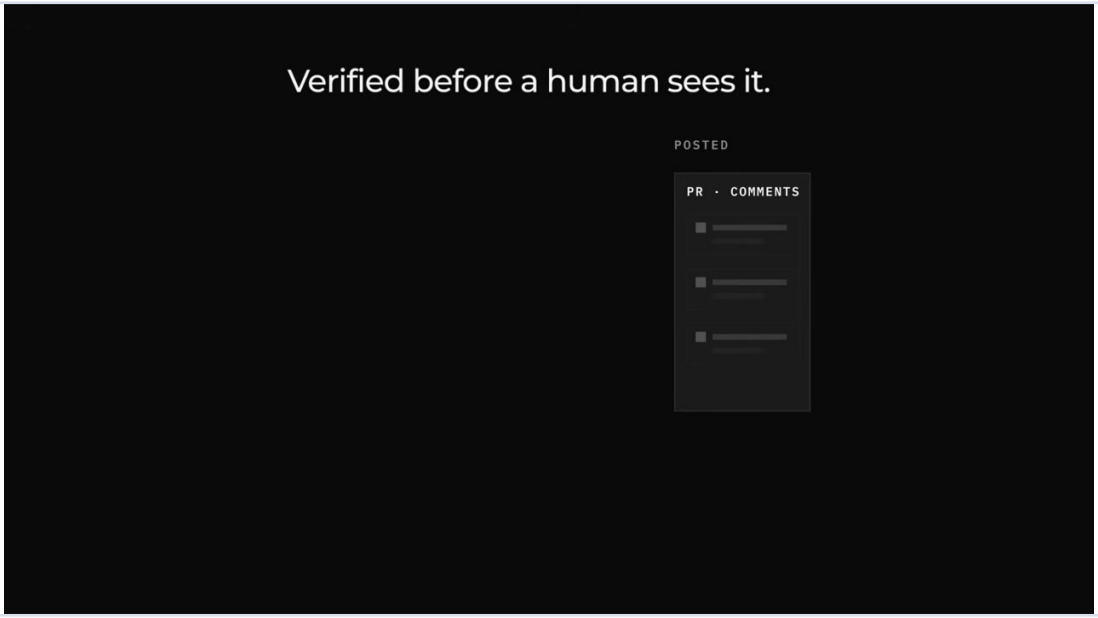
NOTES

- AI review can produce many low-value style comments and flag unchanged code pulled into context.
- A cap limits nitpicks and false positives so high-severity findings remain visible.
- CLAUDE.md or AGENTS.md documents repository conventions; REVIEW.md tells reviewers what matters.
- Repeated findings should become repository guidance or automated checks instead of recurring comments.

CORE IDEA

Tune review noise at the source so important findings retain attention.

Layer 2: parallel finders, a vote, then comments



NOTES

CORE IDEA

Separate discovery from verification before findings reach humans.

Layer 3: security findings are a hard merge gate

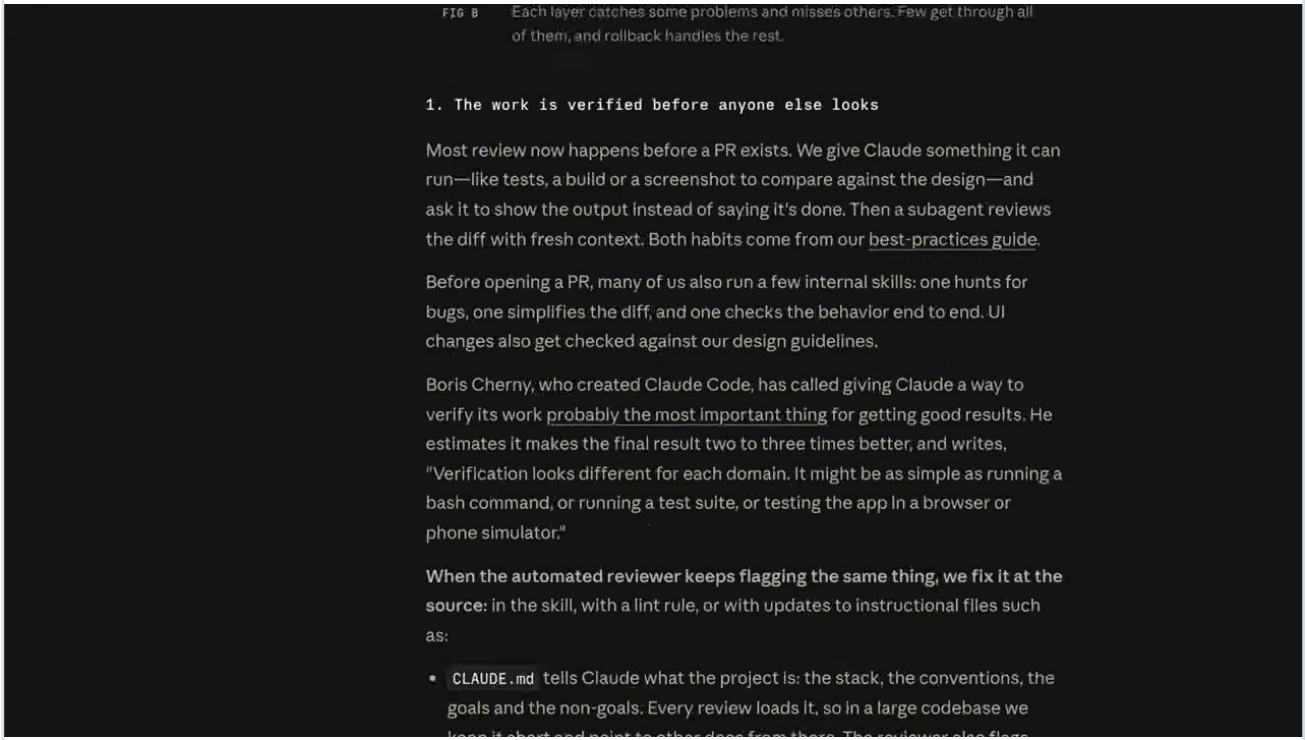


FIG 8 Each layer catches some problems and misses others. Few get through all of them, and rollback handles the rest.

1. The work is verified before anyone else looks

Most review now happens before a PR exists. We give Claude something it can run—like tests, a build or a screenshot to compare against the design—and ask it to show the output instead of saying it's done. Then a subagent reviews the diff with fresh context. Both habits come from our best-practices guide.

Before opening a PR, many of us also run a few internal skills: one hunts for bugs, one simplifies the diff, and one checks the behavior end to end. UI changes also get checked against our design guidelines.

Boris Cherny, who created Claude Code, has called giving Claude a way to verify its work probably the most important thing for getting good results. He estimates it makes the final result two to three times better, and writes, "Verification looks different for each domain. It might be as simple as running a bash command, or running a test suite, or testing the app in a browser or phone simulator."

When the automated reviewer keeps flagging the same thing, we fix it at the source: in the skill, with a lint rule, or with updates to instructional files such as:

- `CLAUDE.md` tells Claude what the project is: the stack, the conventions, the goals and the non-goals. Every review loads it, so in a large codebase we

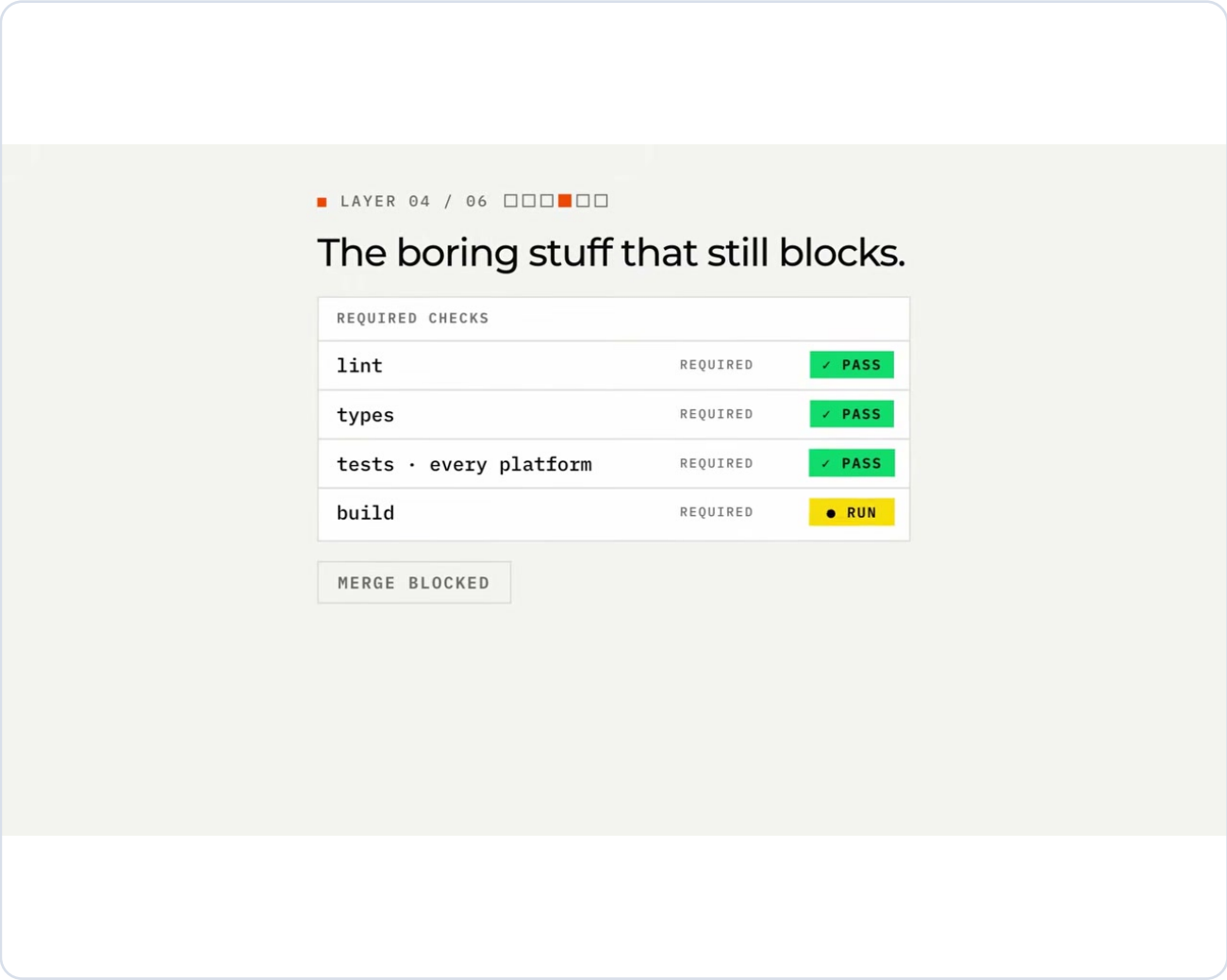
NOTES

- A dedicated security review runs alongside general bug review.
- On Anthropic's own repositories, a pull request cannot merge while either review has an open finding.
- Separating security from general review gives high-impact risks their own policy and ownership.
- Hard gates require reliable triage paths so false positives do not permanently block delivery.

CORE IDEA

Treat unresolved security findings as blockers, not suggestions.

Layer 4: conventional CI still blocks the merge



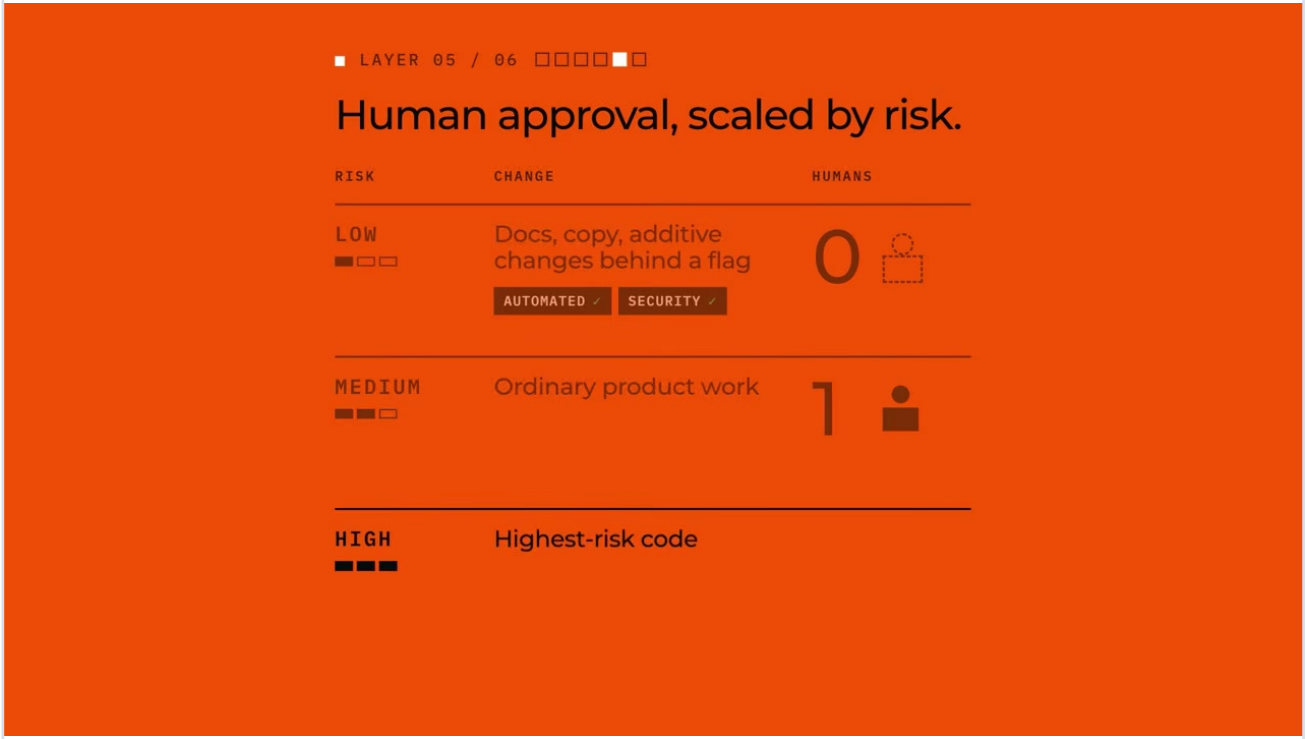
NOTES

- Required checks include linting, type checks, tests on every platform, and a successful build.
- The pull-request template records how the change was tested and how it can be undone.
- AI review supplements these deterministic checks rather than replacing them.
- A green check suite is necessary but cannot prove that product behavior or risk classification is correct.

CORE IDEA

Keep deterministic CI mandatory even when AI review is strong.

Layer 5: scale human approval with risk



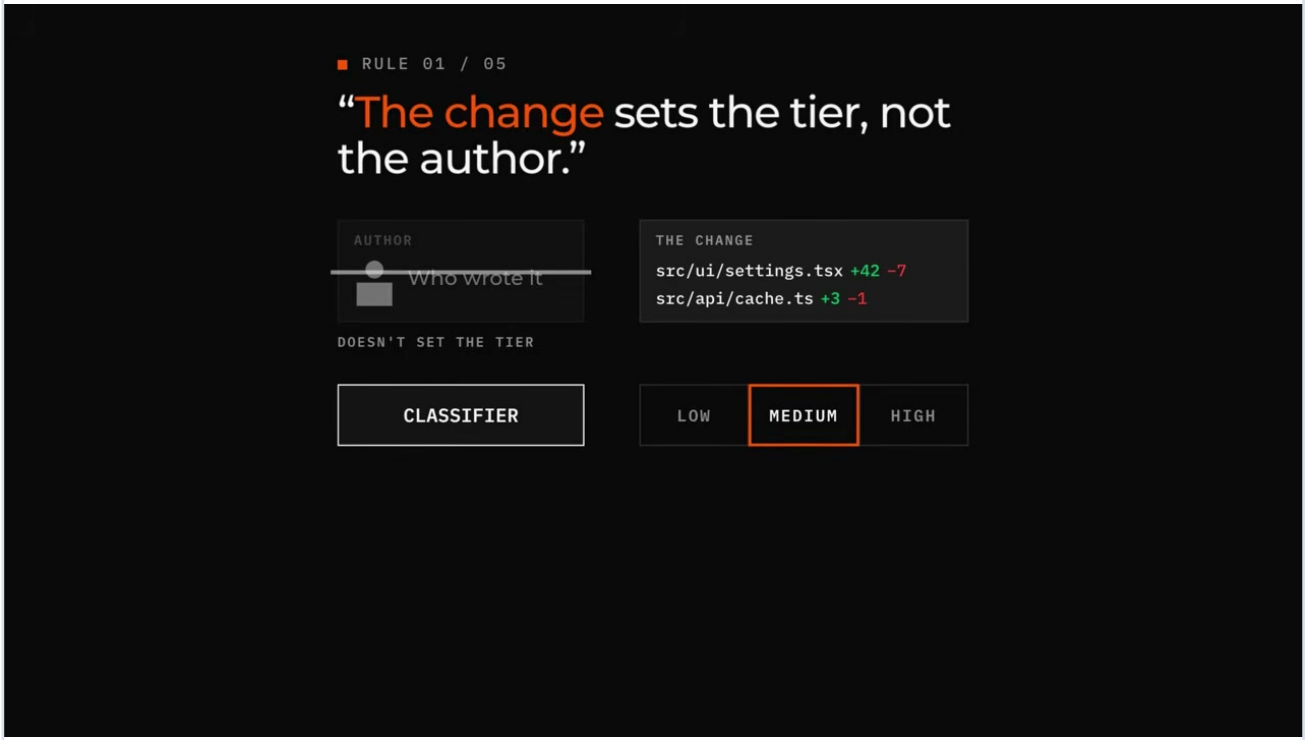
NOTES

- Low-risk changes such as docs or flagged additive work can proceed with zero human approvals after automated checks.
- Ordinary product work requires one human; the highest-risk code requires two.
- The tier should reflect potential blast radius rather than confidence in the author or model.
- Zero-human paths are safe only when classification, monitoring, and rollback are trustworthy.

CORE IDEA

Spend human attention in proportion to possible damage.

Rule 1: the change sets the tier



NOTES

- A classifier inspects the actual change and assigns a low, medium, or high risk tier.
- A second decision checks whether a person is required despite the nominal tier.
- Sensitive files such as CI workflow definitions always require human review.
- Authorship is weak evidence: a small human edit can be dangerous and a large agent edit can be routine.

CORE IDEA

Classify the proposed change, not the identity of its author.

Rule 2: lighter paths require trusted authors



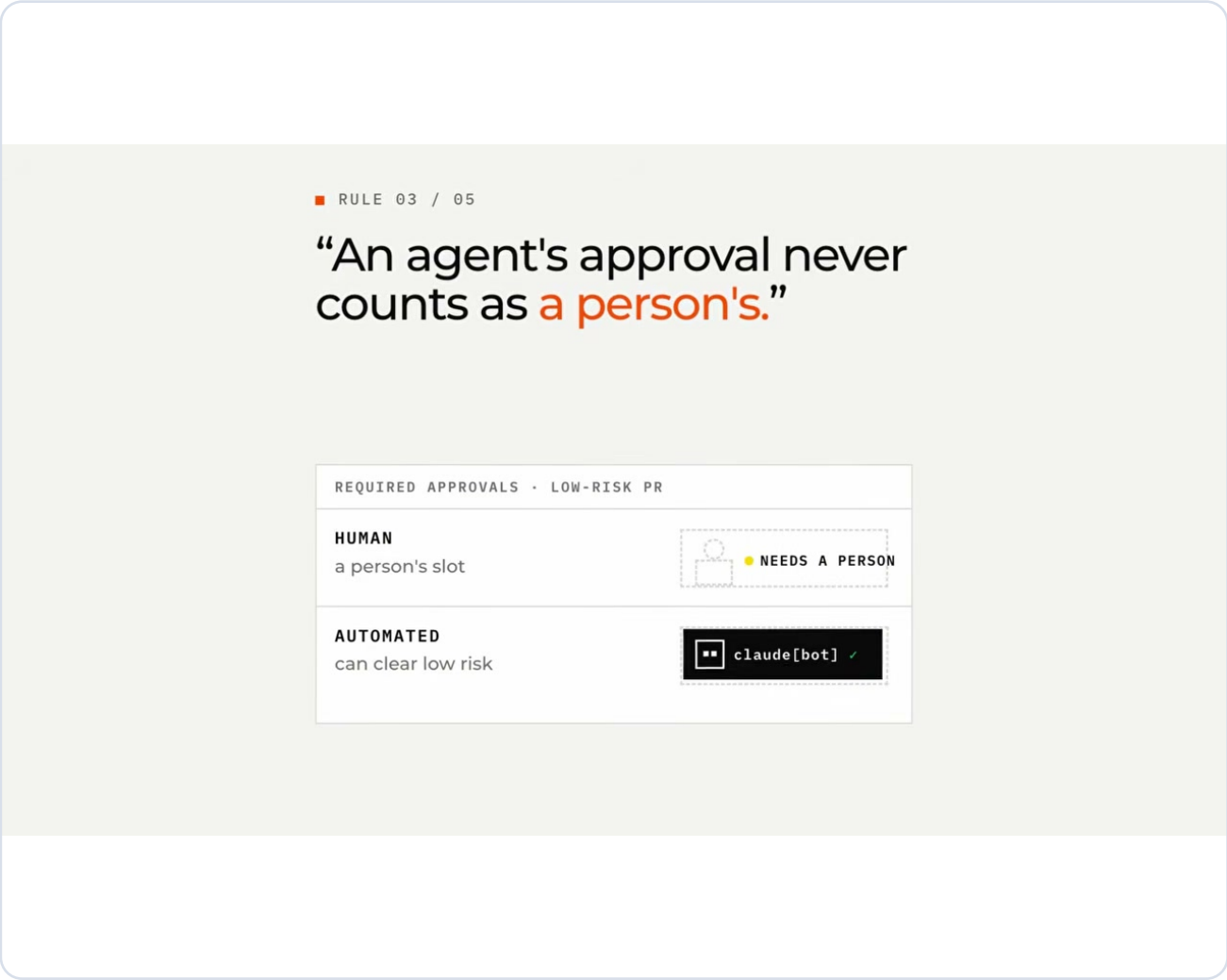
NOTES

- Fast paths are limited to trusted authors, usually members of the team that owns the code.
- Trust controls who may use reduced-friction workflows; it does not redefine the technical risk of a change.
- Ownership knowledge helps identify local hazards that generic classifiers may miss.
- Trust must be explicit and revocable rather than inferred from activity volume.

CORE IDEA

Use lighter paths only for accountable, trusted maintainers.

Rule 3: an agent approval never counts as a person



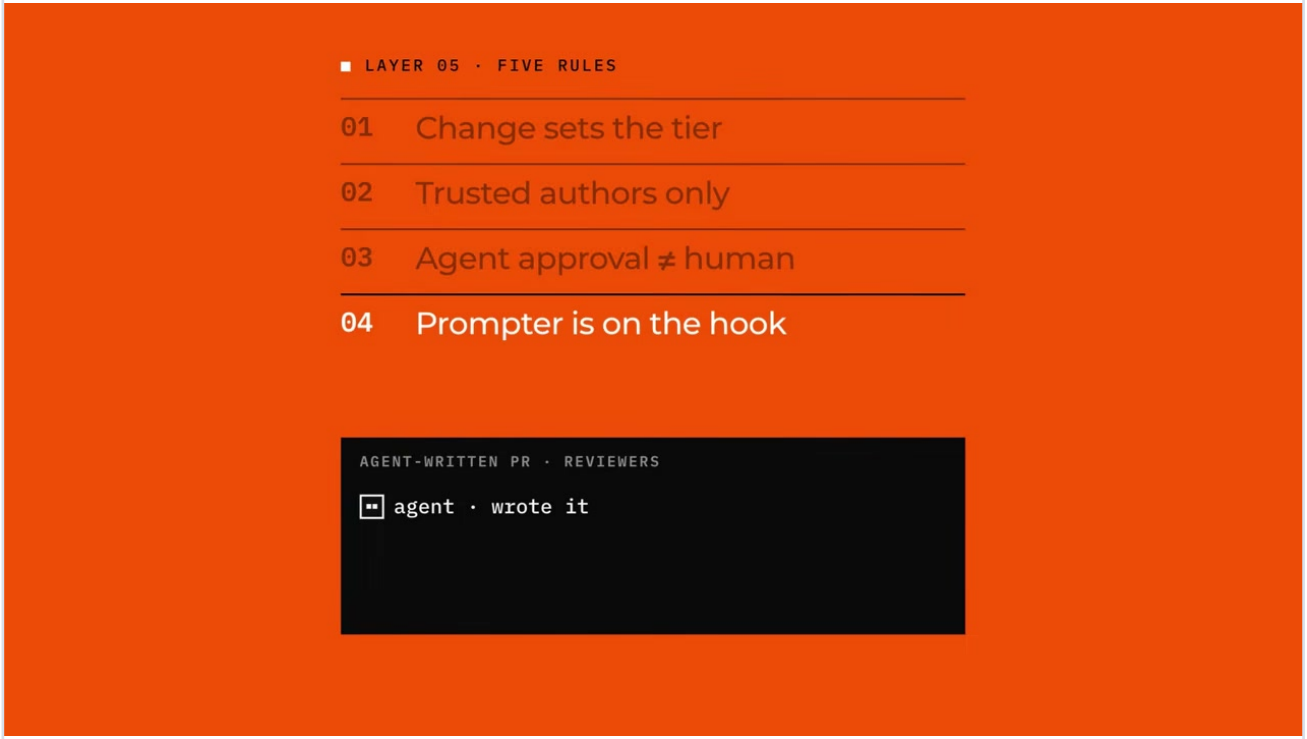
NOTES

- Automated review can satisfy an automated slot but never fills a required human approval slot.
- This prevents several agents from simulating independent human accountability.
- Low-risk changes may need no person only because policy sets the human requirement to zero.
- For higher tiers, a named person must still approve the exact change.

CORE IDEA

Automation may reduce required humans; it cannot impersonate one.

Rule 4: the prompter owns the outcome



NOTES

- The person who directs an agent is accountable for the resulting pull request.
- Responsibility does not disappear when many agents participate in implementation and review.
- A bad result indicates the factory, instructions, or assigned review level was inadequate.
- Accountability requires traceable prompts, owners, and approval records.

CORE IDEA

Keep a named person accountable for agent-generated changes.

Rule 5: approval covers one commit



NOTES

- Approval is bound to a specific commit hash rather than the pull request in general.
- Any new push invalidates the previous approval and triggers another review decision.
- This closes the gap where approved code is modified immediately before merge.
- The same rule should apply to both human-authored and agent-authored follow-up commits.

CORE IDEA

Re-approve after every change to the reviewed commit.

Layer 6: merge queues test combined changes



NOTES

- Two pull requests may pass independently yet fail when combined on the current main branch.
- The merge queue re-runs required checks against the actual integration order.
- This becomes essential when many agents modify related code concurrently.
- Queue throughput must be monitored so the safety layer does not recreate the original bottleneck.

CORE IDEA

Test changes together before merge so main stays green.

Roll out behind flags in widening stages



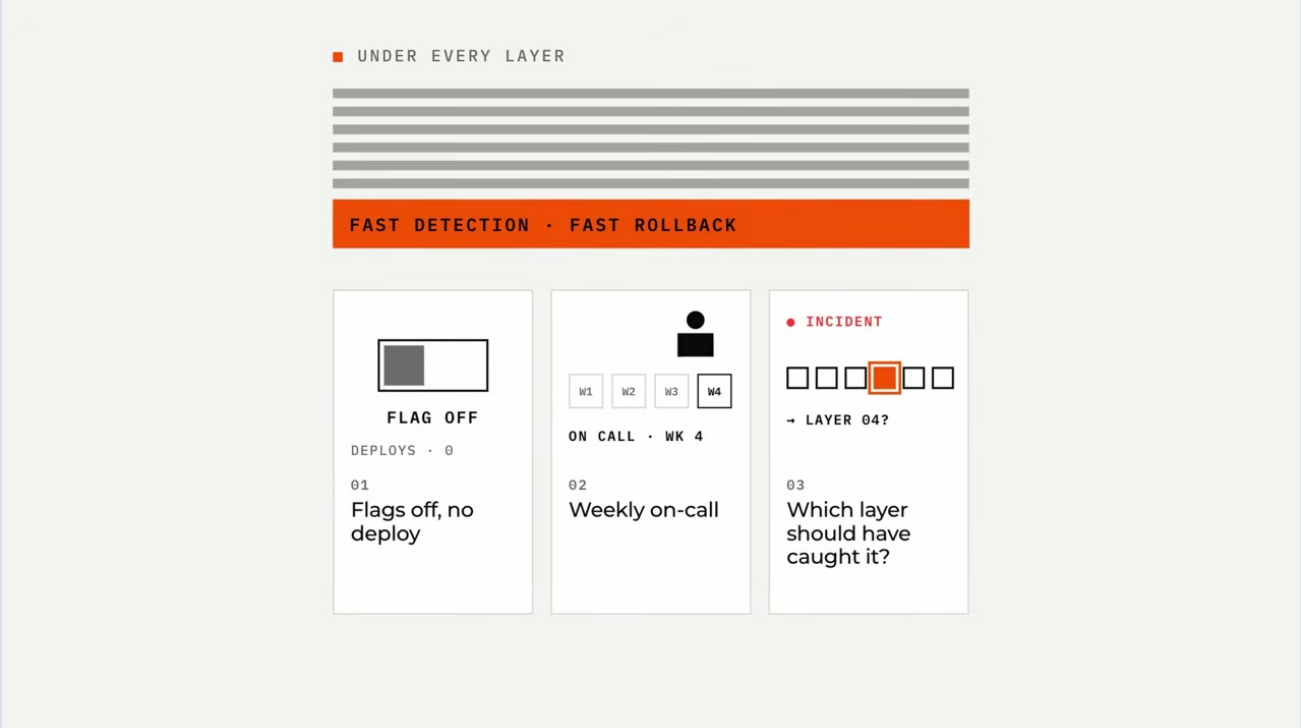
NOTES

- New behavior ships behind a feature flag and reaches Anthropic staff first.
- It then progresses through a broader latest channel before reaching stable users.
- Each stage limits exposure while producing real operating evidence.
- A staged rollout is valuable only when teams can disable the flag quickly and observe meaningful signals.

CORE IDEA

Increase exposure gradually and preserve a fast off switch.

Fast detection and rollback support every layer



NOTES

- Feature flags can disable faulty behavior without waiting for a new deployment.
- A weekly on-call rotation provides a named response owner.
- Each incident review asks which layer should have caught the failure, turning incidents into control improvements.
- Recovery speed is a system property: flags, telemetry, ownership, and practice must all work together.

CORE IDEA

Improve the review system after every incident, not just the broken code.

The missing circuit breaker



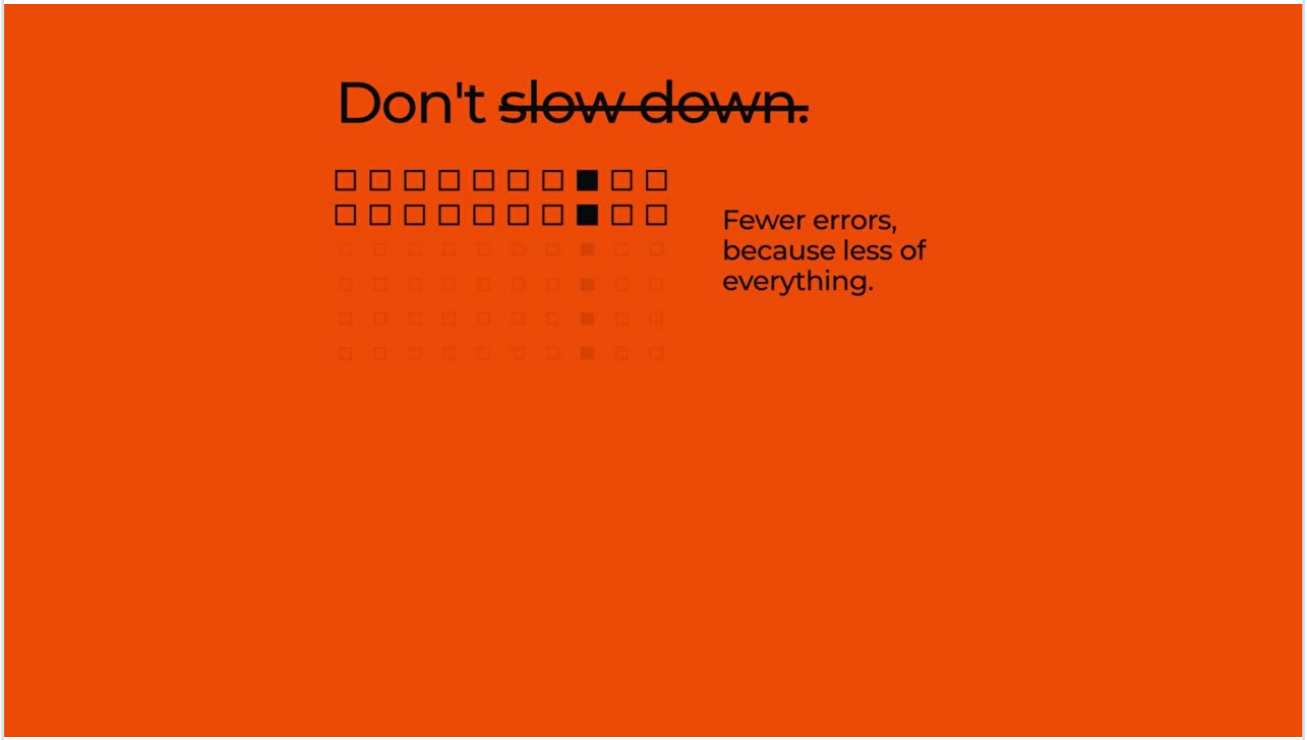
NOTES

- The process lacks an automatic slowdown when reverts or incidents begin rising.
- The bottling-line analogy shows how a defect at an early station can spoil many downstream units.
- A circuit breaker would reduce throughput or stop risky paths when guardrails breach a threshold.
- Thresholds must avoid both late intervention and frequent false alarms.

CORE IDEA

Automatically slow the pipeline when safety signals deteriorate.

The operating thesis: get faster at fixing



NOTES

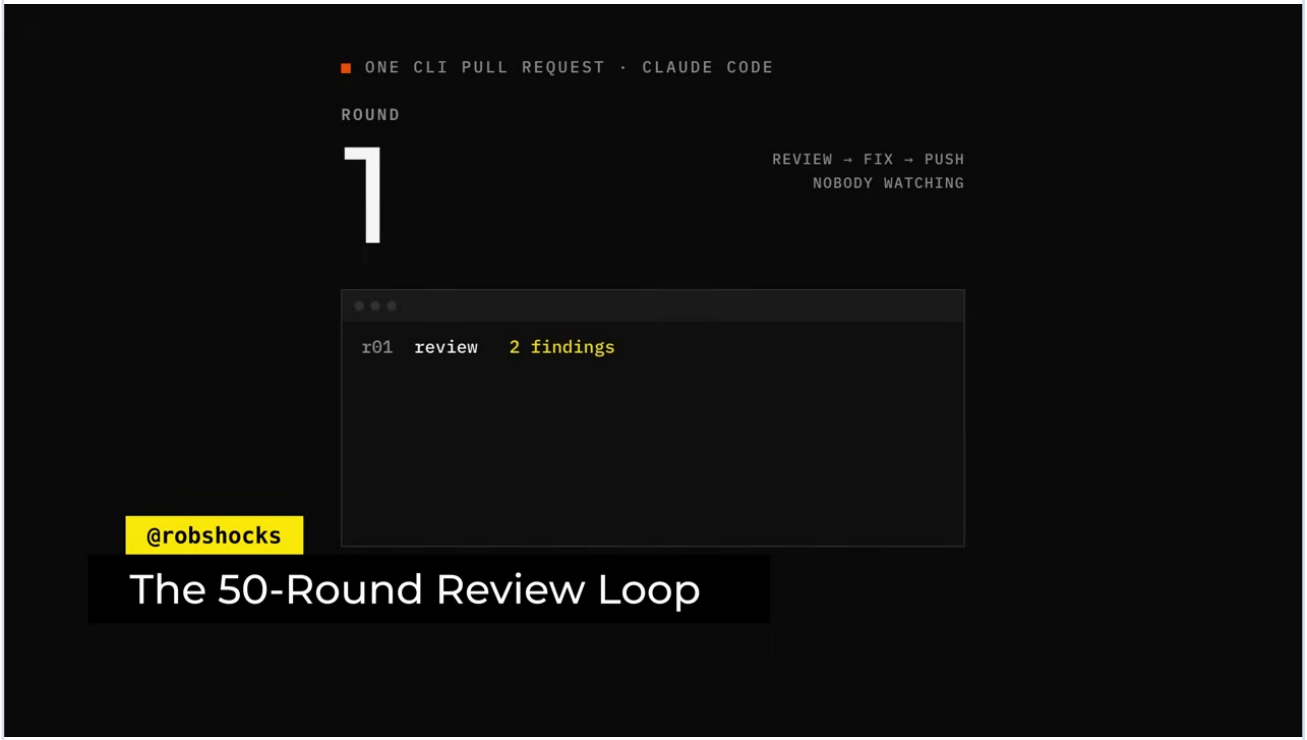
- Simply reducing output lowers errors partly because less work is attempted.
- The preferred strategy is to shorten the time from detection to repair and recovery.
- That strategy depends on strong telemetry, staged rollout, ownership, and rollback—not speed alone.
- Anthropic benefits from greater inference usage, so this framing carries a commercial interest.

CORE IDEA

Optimize detection and recovery, not raw output at any cost.

An unattended pull request exceeded 50 review rounds

Self-reported failure story



NOTES

- One Claude Code CLI pull request reportedly repeated review, fix, push, and re-review more than 50 times overnight.
- No person was present to dismiss low-value findings or stop the loop.
- Each pass found a few additional issues, creating more commits and another review cycle.
- The example exposes a control failure, not proof that autonomous review always behaves this way.

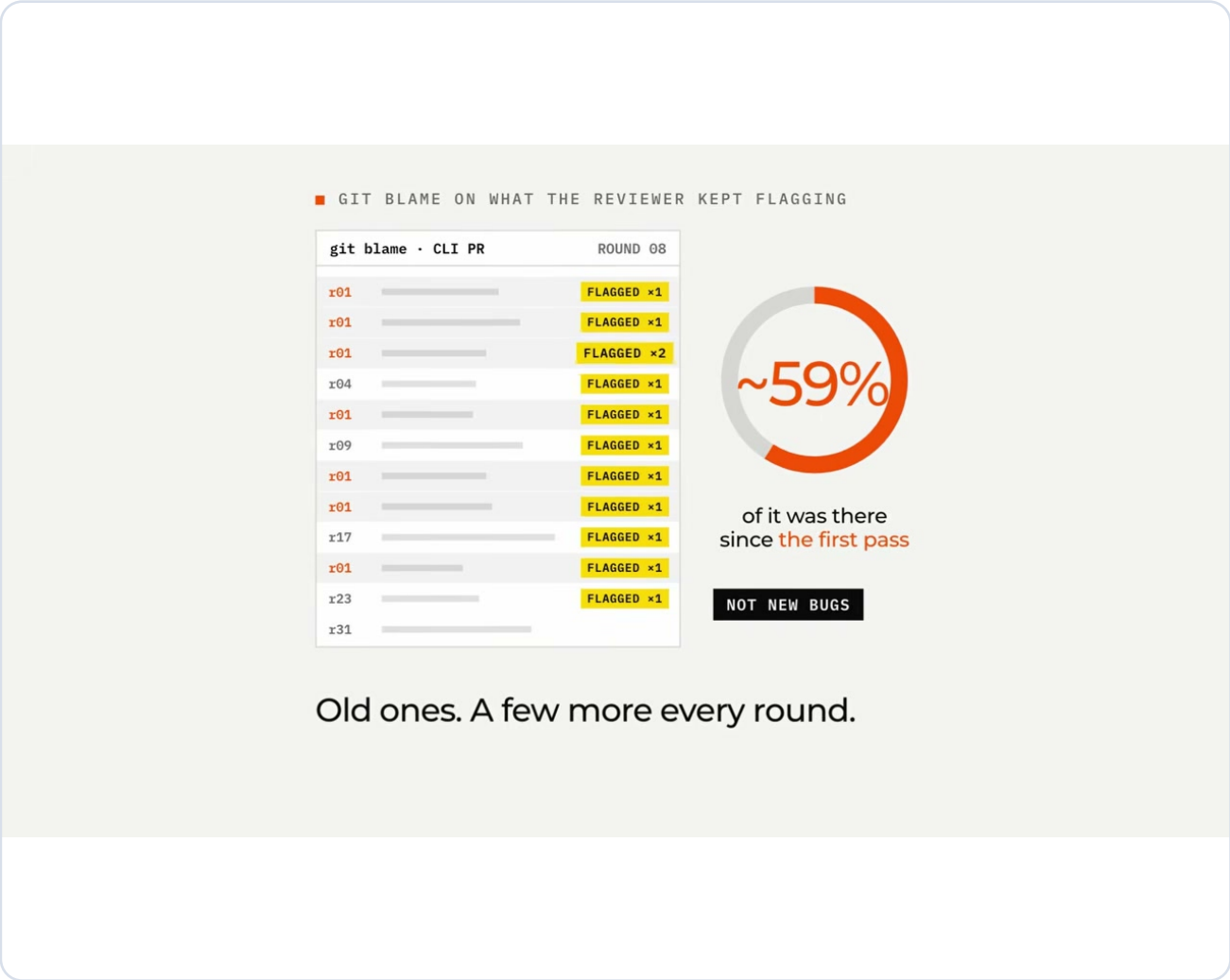
CORE IDEA

Autonomous review loops need explicit stopping conditions.

Source: https://www.youtube.com/watch?v=bls8ZJ_INMQ

Most repeated findings were already in the first pass

~59% — self-reported



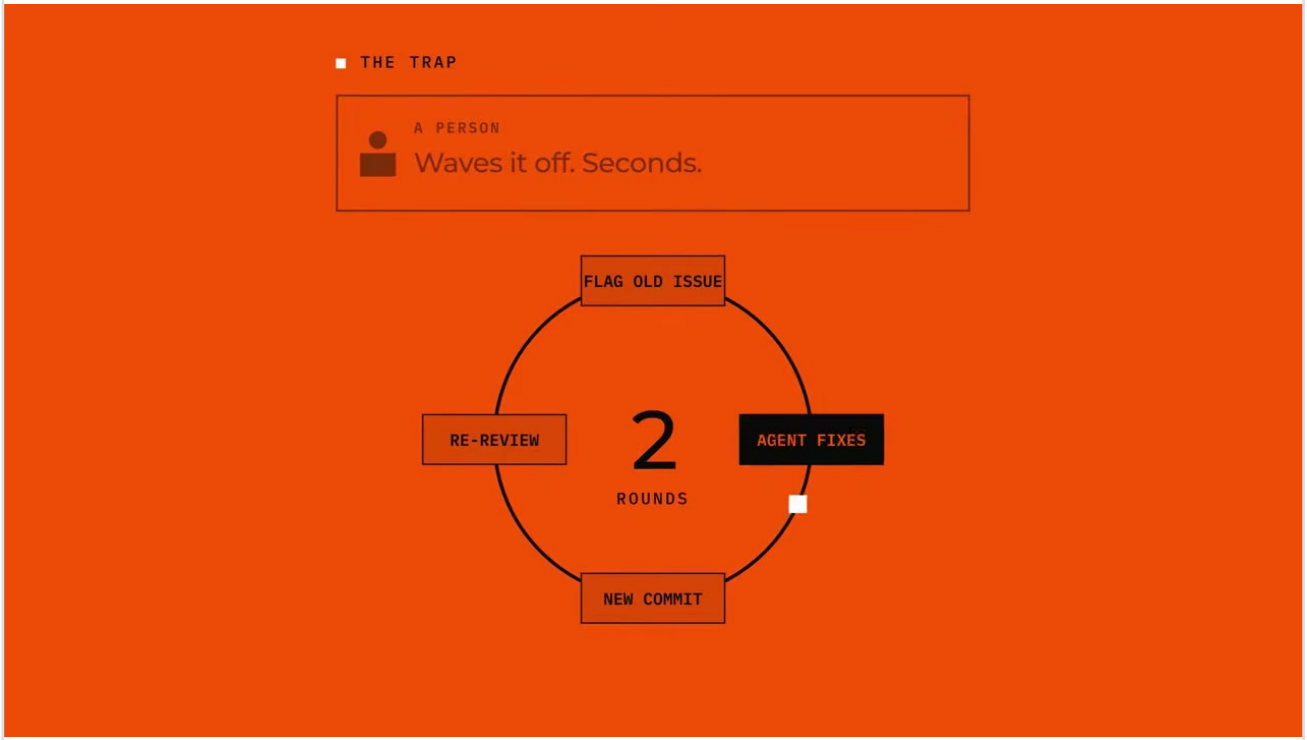
NOTES

- Git blame traced the code that the reviewer repeatedly flagged across rounds.
- About 59% of those issues reportedly existed since the first pass.
- The model was not creating new bugs; it was progressively discovering older ones.
- Finding more issues can still be counterproductive when the loop lacks priority, scope, and a stopping rule.

CORE IDEA

More findings are not always more value.

The trap: agents act on findings people dismiss



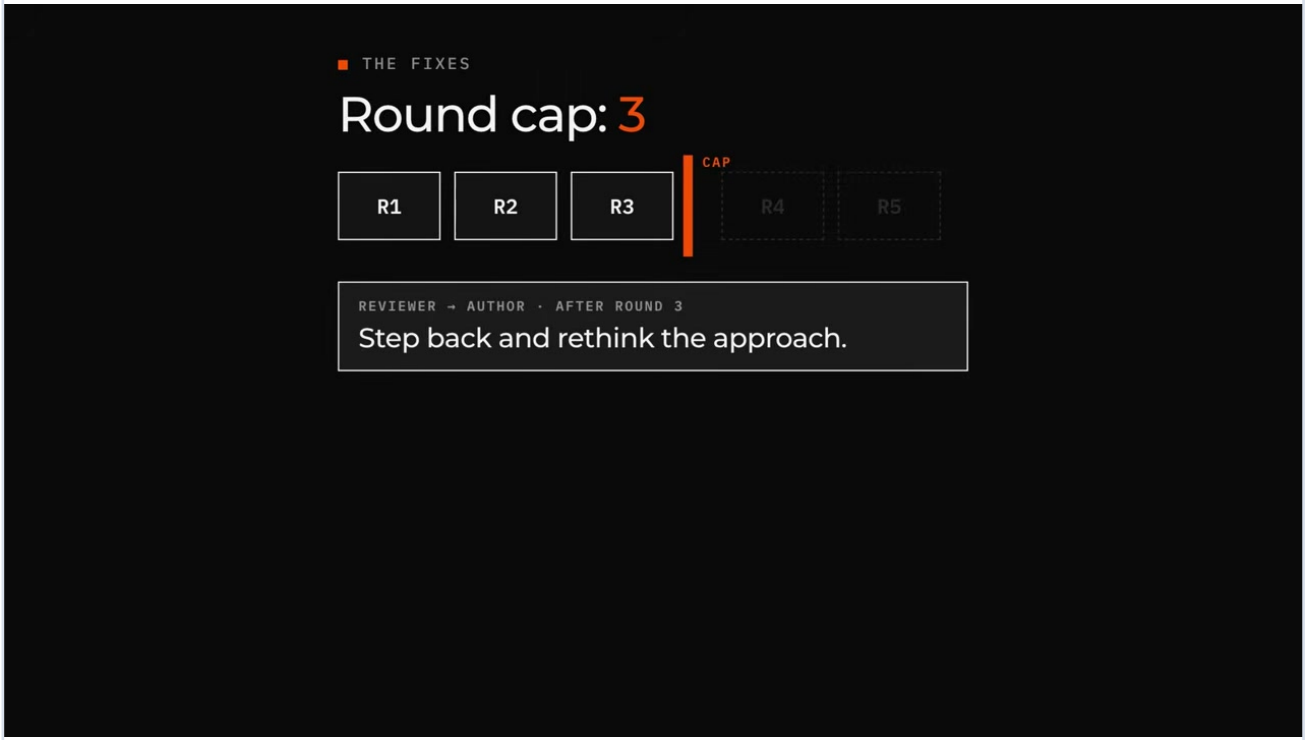
NOTES

- A person can dismiss an irrelevant nitpick in seconds.
- An agent tends to fix every reported issue, producing another commit and another review.
- Even harmless findings become costly when they trigger a repeated fix-and-review cycle.
- Review policies must distinguish required defects from optional improvements.

CORE IDEA

Teach agents when not to act, not only how to fix.

Cap rounds and narrow later reviews



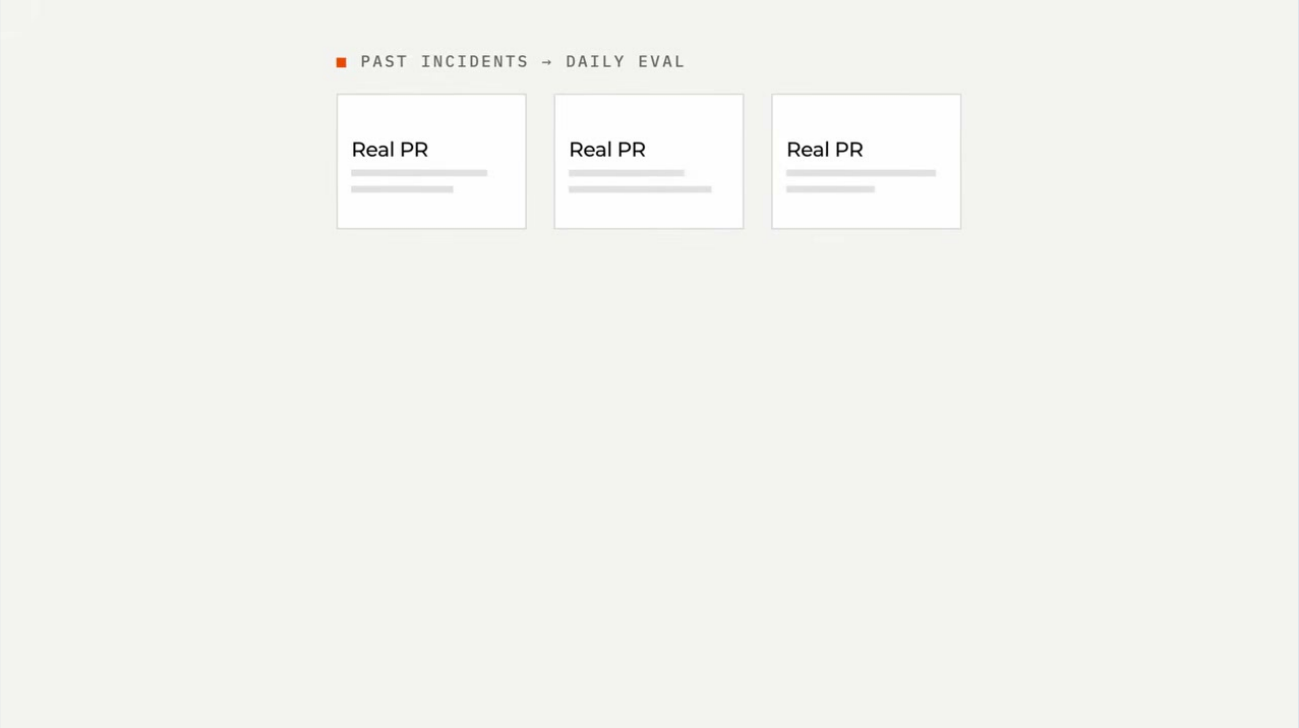
NOTES

- After three rounds, the workflow stops and asks for a rethink instead of continuing automatically.
- Later passes review only the new diff rather than reopening every earlier concern.
- The reviewer flags the whole bug class and never repeats the same finding.
- Round limits trade some possible discoveries for bounded cost and predictable completion.

CORE IDEA

Bound the loop, narrow its scope, and prevent repeated findings.

Real incidents become a daily evaluation set



NOTES

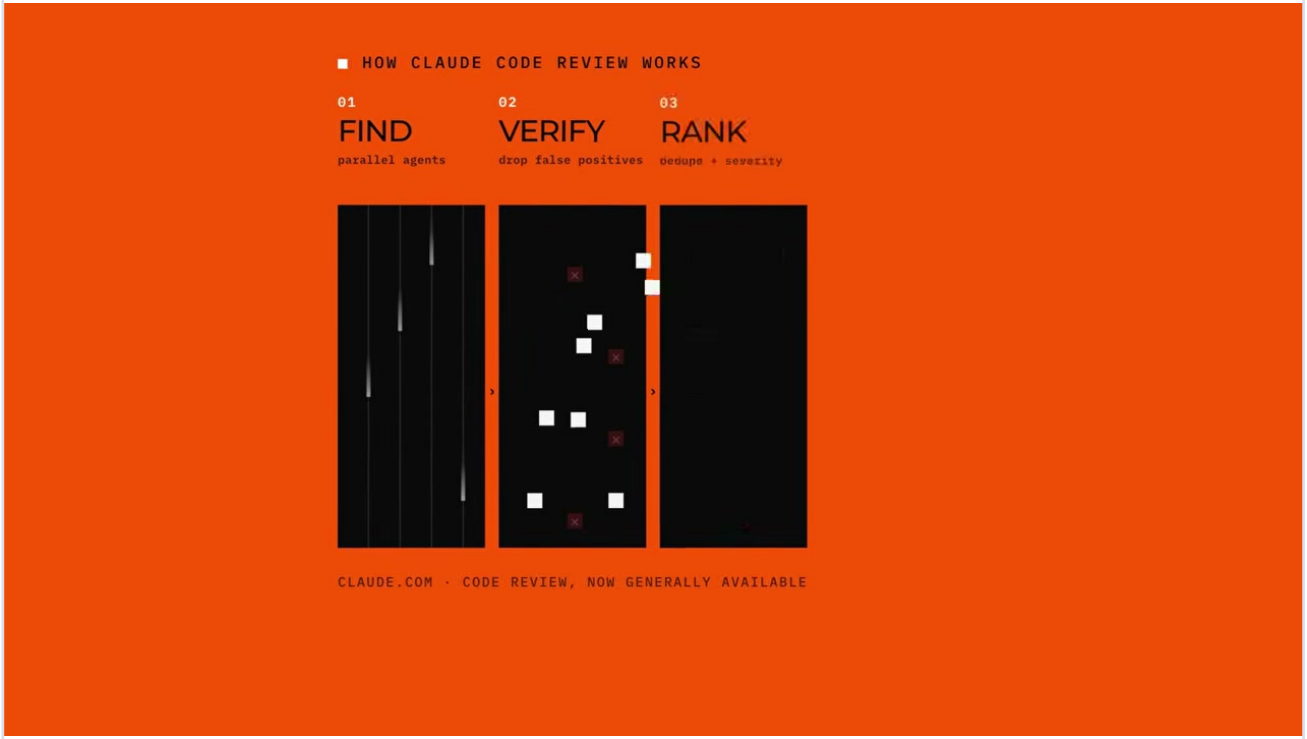
- Engineers confirm pull requests that caused real incidents and add them to a known-bad evaluation set.
- The reviewer runs against this set daily to detect improvement or regression.
- Production failures therefore become durable tests rather than one-time lessons.
- No internal evaluation results are shown, so effectiveness cannot be independently assessed.

CORE IDEA

Turn costly incidents into permanent tests of reviewer quality.

Claude Code Review: find, verify, rank, post

Public product workflow — vendor description



NOTES

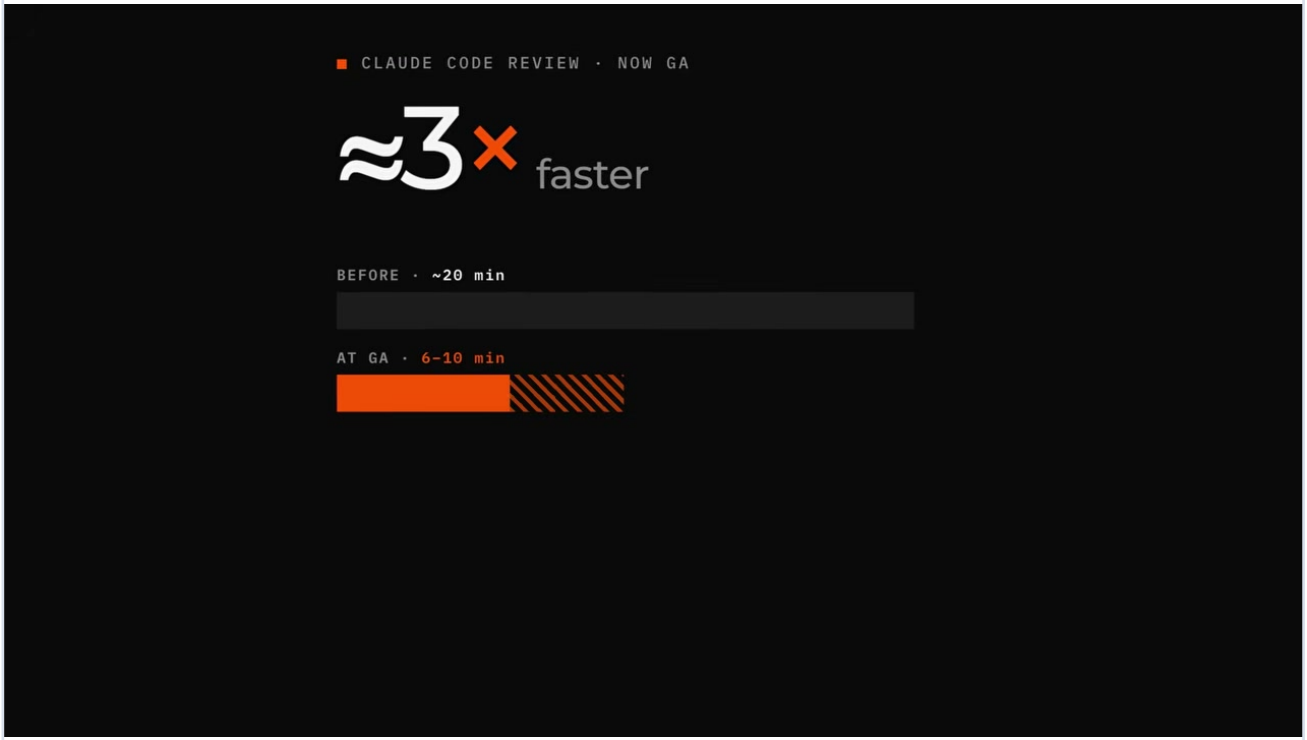
- Parallel agents find candidate issues in the diff.
- A verification step removes false positives before findings are ranked by severity.
- Results appear as inline comments and a pull-request check run.
- This is a product description and has not been independently benchmarked here.

CORE IDEA

Productize discovery and verification before posting review comments.

Review is claimed to be approximately 3× faster

Vendor-reported performance



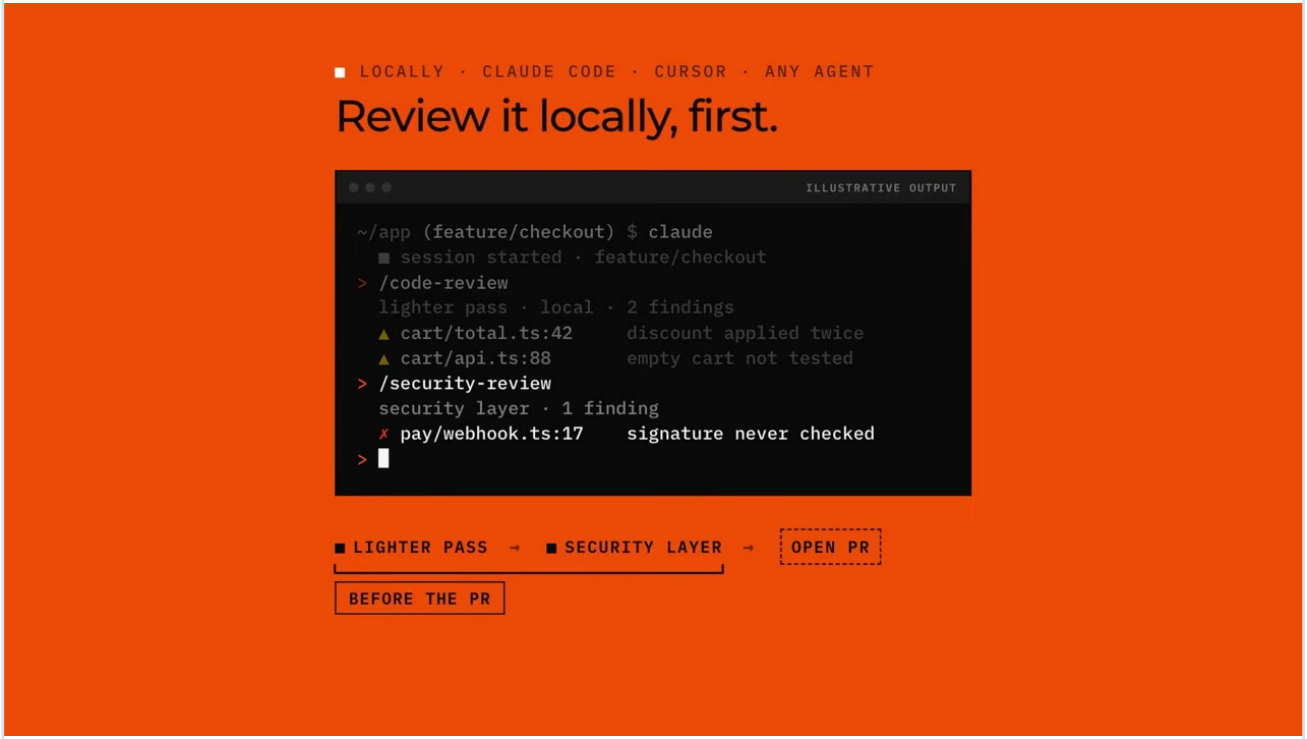
NOTES

- Anthropic reports review time falling from roughly 20 minutes to 6–10 minutes.
- Repositories can choose quick, standard, or deep review, with deeper analysis costing more.
- The product reports findings but does not approve or block a pull request by itself.
- Treat the speed claim as vendor evidence until reproduced on representative repositories.

CORE IDEA

Choose review depth deliberately and validate vendor speed claims locally.

Run review locally before opening the pull request



NOTES

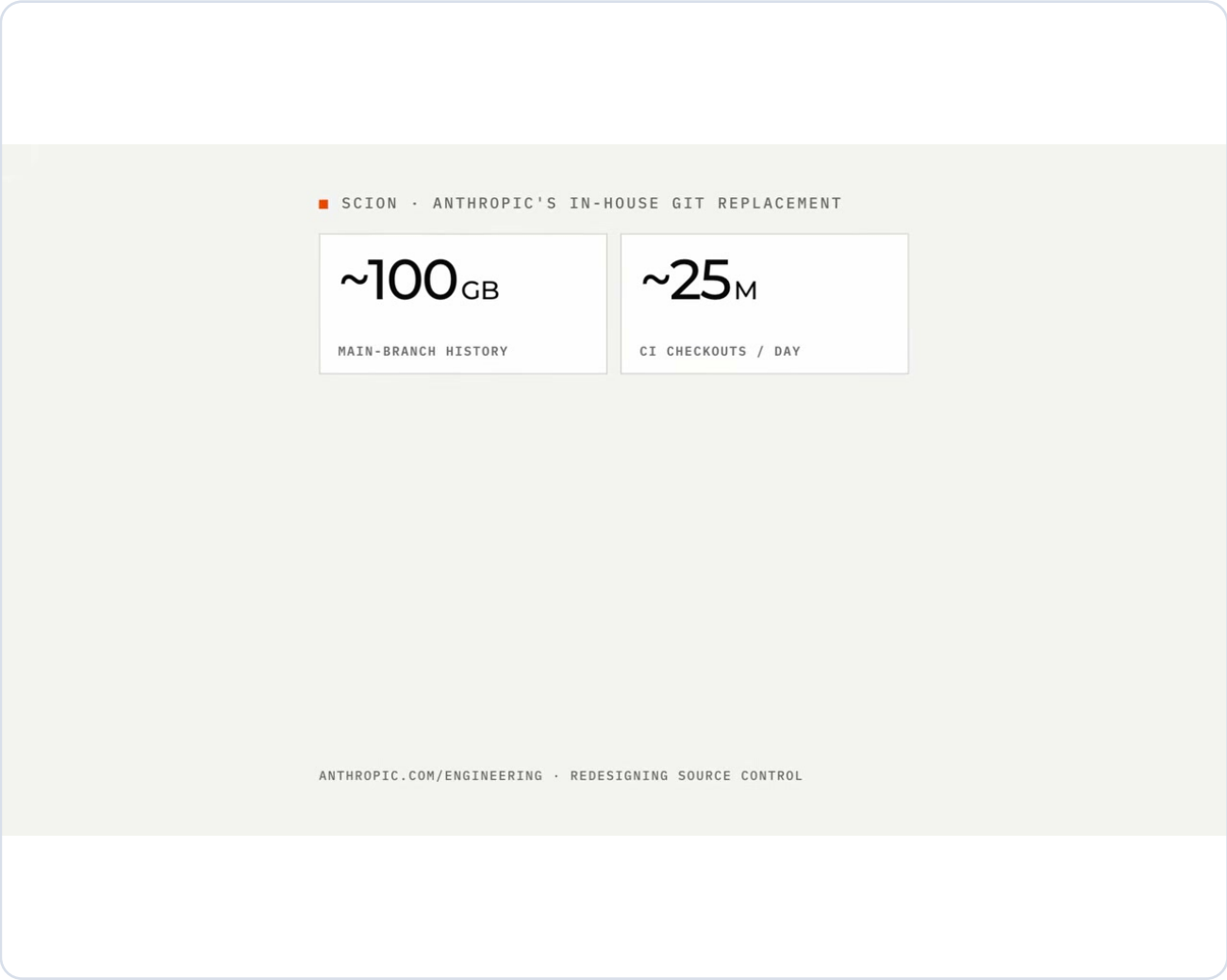
- A local code-review command examines the feature branch before remote review begins.
- The same pattern can be implemented with other coding agents and editors.
- A lightweight local pass removes obvious issues before security checks and pull-request review.
- Local review should not bypass repository policy or hide material findings from collaborators.

CORE IDEA

Catch cheap issues locally before they enter the shared review queue.

Scion replaces Git for Anthropic's monorepo scale

Self-reported infrastructure claims



NOTES

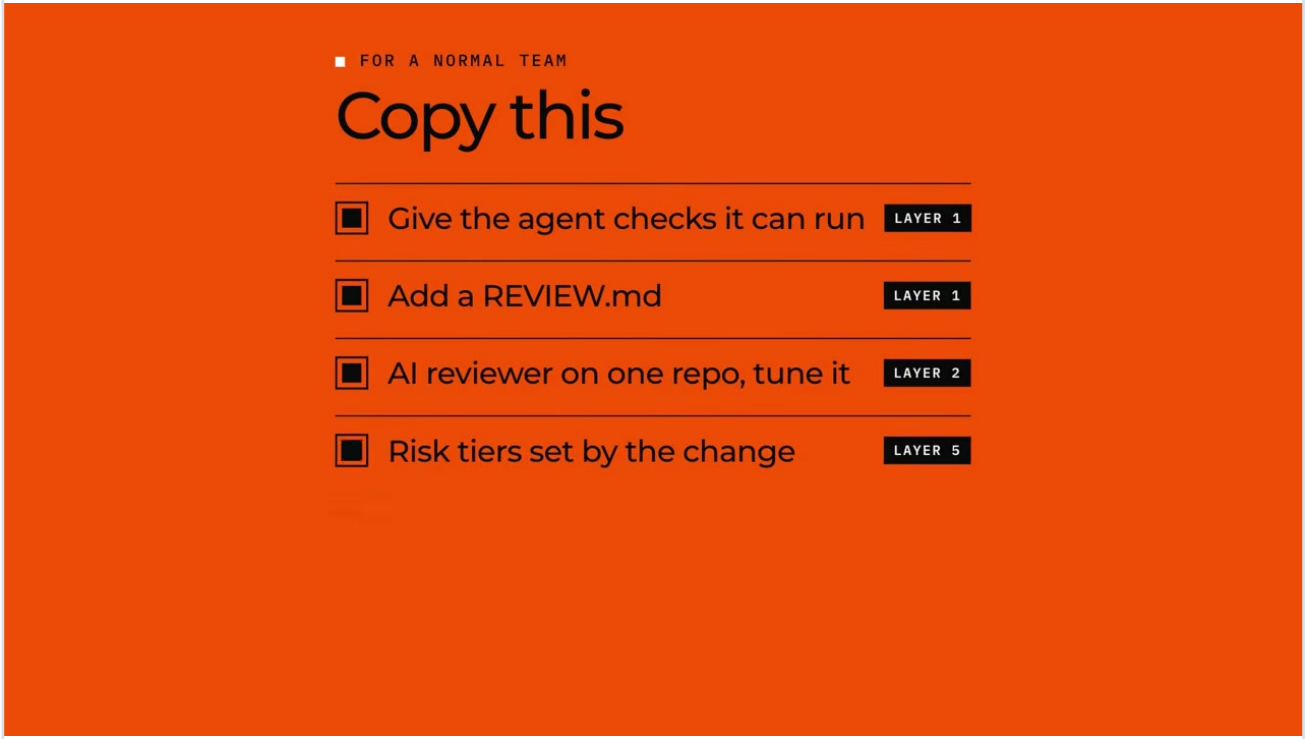
- The repository reportedly contains nearly 100 GB of main-branch history and serves about 25 million CI checkouts daily.
- A seven-person team built Scion and migrated the organization in roughly five months.
- The redesign addresses source-control scale rather than ordinary code-review policy.
- The results are not independently tested outside Anthropic and are irrelevant to most smaller repositories.

CORE IDEA

Replace core infrastructure only when scale makes the existing tool a measured bottleneck.

Source: https://www.youtube.com/watch?v=bls8ZJ_INMQ

A practical adoption sequence for normal teams



NOTES

- Give agents executable checks, then document review priorities in REVIEW.md.
- Start with one repository and tune the AI reviewer against real findings for several weeks.
- Set risk tiers from the change, never let bot approval fill a human slot, and track reverts as a guardrail.
- This sequence is a practical recommendation, not an Anthropic-published standard.

CORE IDEA

Start small, encode expectations, tune on real work, and monitor reverts.